

Guía de Estudio: TP1 de Aprendizaje Automático 1

Predicción de precios de casas con regresión lineal

Tecnicatura Universitaria en Inteligencia Artificial · FCEIA, UNR ·
Segundo cuatrimestre 2026

Esta guía explica el trabajo práctico completo tal como quedó en el notebook [TP-regresion-AA1.ipynb](#) del repositorio (versión de GitHub del 28/09/2026). Todos los números que aparecen salen de las salidas reales del notebook. Los gráficos marcados como "ejemplo inventado" usan datos de juguete para que el concepto se vea más claro; los marcados como "datos reales" o "figura del notebook" son del TP.

Está pensada para alguien que todavía no tiene todos los conceptos firmes. Por eso avanza de lo más simple a lo más complejo, explica las partes del código que más suelen preguntar en la defensa (por ejemplo, la diferencia entre `.fit()` y `.fit_transform()`) y cierra con preguntas para que te autoevalúes.

Contenido

Objetivo de Aprendizaje	3
Conceptos Clave (El "Qué" y el "Por qué")	3
Desarrollo Estructurado	5
1. El problema en una página	5
2. Lo mínimo de Python para leer el notebook	6
3. La pregunta estrella: fit, transform, fit_transform y predict	7
4. Limpieza de los datos	8
5. Análisis exploratorio	9
6. Outliers: ¿error o caso raro pero real?	12
7. Separar train y test (y por qué antes de escalar e imputar)	12
8. Escalado	13

9. Imputación de valores faltantes	13
10. Regresión lineal	16
11. Descenso del gradiente	17
12. Métricas: cómo medimos el error	19
13. Regularización: Ridge, Lasso y ElasticNet	21
14. Optimización de hiperparámetros	22
15. Comparación final y conclusión	25
16. Preguntas sobre el código que te pueden hacer	25
Ejemplo Práctico / Analogía	28
Autoevaluación Rápida	28

Objetivo de Aprendizaje

Al terminar esta guía vas a poder explicar, con tus palabras y con el código del notebook a la vista, cada paso del TP: cómo se limpiaron y prepararon los datos sin filtrar información de test, cómo funcionan la regresión lineal, el descenso del gradiente y la regularización, cómo se eligieron los hiperparámetros y por qué el modelo final es LinearRegression.

Conceptos Clave (El "Qué" y el "Por qué")

Definiciones cortas para tener a mano. Cada una se desarrolla más adelante.

Datos y preparación

- **Target (variable objetivo):** lo que queremos predecir. Acá es **MEDV**, el valor mediano de las viviendas de un barrio en miles de dólares. Las otras 13 columnas son las **features** (variables de entrada).
- **Regresión:** problema de aprendizaje supervisado donde el target es un número continuo (un precio). Si fuera una categoría (sí/no) sería clasificación.
- **Train y test:** dividimos los datos en dos partes. Con **train** (80 %) el modelo aprende; con **test** (20 %) medimos qué tan bien predice datos que nunca vio. Test simula "las casas del futuro".
- **Fuga de datos (data leakage):** cuando información de test se cuela en el entrenamiento o en el preprocesamiento. Hace que las métricas de test salgan mejores de lo que en realidad son.
- **fit, transform, fit_transform, predict:** los cuatro verbos de scikit-learn. **fit** aprende algo de los datos, **transform** aplica lo aprendido, **fit_transform** hace las dos cosas sobre los mismos datos y **predict** usa un modelo entrenado para predecir el target.
- **Valor faltante (nulo, NaN):** celda vacía. Se puede eliminar la fila o **imputar** (rellenar) el valor.
- **Outlier:** valor muy alejado del resto. Puede ser un error de carga o un caso real pero raro.
- **IQR (rango intercuartílico):** $Q3 - Q1$, el ancho del 50 % central de los datos. Lo usamos para detectar outliers y para escalar, porque no se deja arrastrar por los extremos.
- **Escalado:** llevar las variables a escalas comparables. Importa porque KNN usa distancias y el descenso del gradiente y la regularización tratan a todas las variables con el mismo número.
- **Correlación:** mide cuánto se mueven juntas dos variables, de -1 a 1. **Pearson** mide la relación lineal; **Spearman** mide si la relación es creciente o decreciente, usando el orden de los datos.
- **Multicolinealidad:** dos features muy correlacionadas entre sí (en el TP, RAD y TAX). El modelo tiene problemas para repartir el efecto entre ellas.

Modelos y entrenamiento

- **Regresión lineal:** predice el target como una suma ponderada de las features más una constante: $\hat{y} = w_0 + w_1x_1 + \dots + w_{13}x_{13}$. Los w son los **coeficientes** y w_0 es el **intercepto**.

- **Parámetro vs hiperparámetro:** los parámetros (los w) los aprende el modelo con **fit**. Los hiperparámetros (tasa de aprendizaje, tamaño de lote, α , k vecinos) los elegimos nosotros antes de entrenar.
- **Función de costo:** número que mide qué tan mal predice el modelo. Usamos el **MSE** (error cuadrático medio). Entrenar es buscar los w que la hacen mínima.
- **Descenso del gradiente:** algoritmo que encuentra el mínimo de la función de costo dando pasos chicos "cuesta abajo". El tamaño del paso es la **tasa de aprendizaje (η)**.
- **Época:** una pasada completa por todas las filas de train.
- **Regularización** (Ridge, Lasso, ElasticNet): agrega al costo un castigo por coeficientes grandes, para evitar el sobreajuste. La fuerza del castigo es **α** .
- **Validación cruzada:** divide train en partes y entrena/valida varias veces, para elegir hiperparámetros sin tocar test.

Evaluación

- **MAE, RMSE y R^2 :** las métricas del TP. MAE y RMSE miden el error en miles de dólares; R^2 es la proporción de la variabilidad del precio que explica el modelo.
- **Sobreajuste (overfitting):** el modelo memoriza train y le va mucho peor en test.
- **Subajuste (underfitting):** el modelo es demasiado simple y le va mal en los dos.
- **Compromiso sesgo-varianza:** un modelo muy simple tiene mucho sesgo (subajusta); uno muy flexible tiene mucha varianza (sobreajusta). La regularización suma sesgo para bajar varianza.

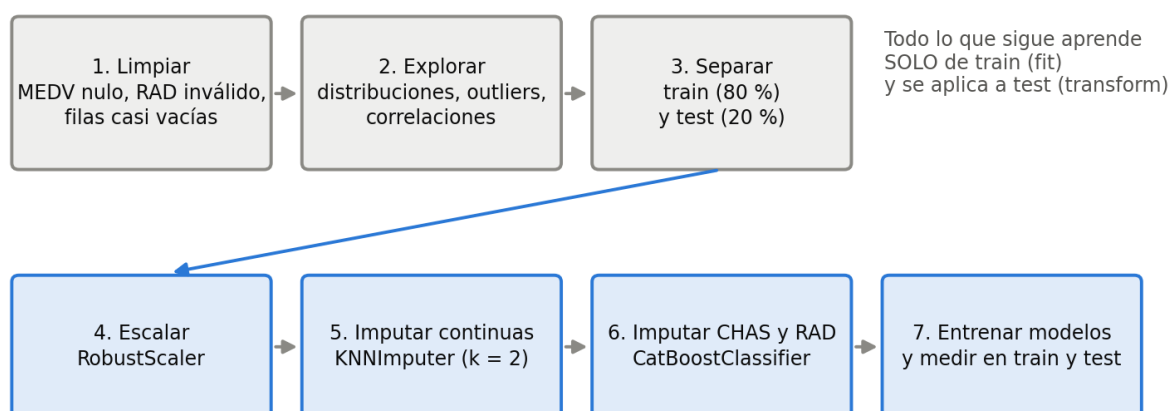
Desarrollo Estructurado

1. El problema en una página

El dataset `house-prices-tp.csv` es una versión del clásico Boston Housing: 556 filas (barrios) y 14 columnas, todas numéricas. Queremos predecir MEDV a partir de las otras 13. A diferencia del original, esta versión viene "sucia": tiene 306 celdas nulas (3.93 % del total) y valores de RAD que no corresponden a ninguna categoría (por ejemplo 10.55 o 14.64).

Variable	Tipo	Qué es
CRIM	continua	tasa de criminalidad per cápita
ZN	continua	proporción de suelo residencial para lotes grandes
INDUS	continua	proporción de acres comerciales no minoristas
CHAS	categoría (0/1)	1 si el barrio limita con el río Charles
NOX	continua	concentración de óxidos nítricos (contaminación)
RM	continua	habitaciones promedio por vivienda
AGE	continua	proporción de viviendas construidas antes de 1940
DIS	continua	distancia ponderada a cinco centros de empleo
RAD	discreta ordinal	índice de acceso a autopistas: {1, ..., 8, 24}
TAX	continua	impuesto a la propiedad por cada 10.000 USD
PTRATIO	continua	alumnos por maestro
B	continua	índice relacionado a la proporción de población afroamericana
LSTAT	continua	% de población de bajo estatus socioeconómico
MEDV	continua (target)	valor mediano de las viviendas, en miles de USD

El orden de los pasos importa tanto como los pasos mismos. Este es el recorrido completo del notebook:



Las filas incoherentes en 2 o más variables se sacan solo de train; test queda intacto.

Gráfico 1. Orden del trabajo. Los pasos grises usan todo el dataset; los azules aprenden solo de train.

La idea que atraviesa todo el TP

Cada vez que algo "aprende" de los datos (un escalador, un imputador, un modelo), aprende de train y después se aplica a test. Si entendés esto, entendés la mitad de las preguntas sobre el código.

2. Lo mínimo de Python para leer el notebook

No hace falta saber programar de memoria, pero sí reconocer qué hace cada línea. Estas son las piezas que más se repiten.

Si ves...	Significa...
<code>df_casas</code>	un DataFrame de pandas: una tabla con filas (índice) y columnas con nombre.
<code>df['RAD']</code>	una columna sola (una Series).
<code>df.isnull().sum()</code>	cuenta los nulos de cada columna (suma por columna, axis=0 por defecto).
<code>df.isnull().sum(axis=1)</code>	cuenta los nulos de cada fila. axis=0 recorre filas hacia abajo (resultado por columna); axis=1 recorre columnas (resultado por fila).
<code>df.dropna(subset=['MEDV'])</code>	borra las filas que tienen NaN en MEDV.
<code>df.dropna(thresh=7)</code>	se queda con las filas que tienen al menos 7 valores no nulos .
<code>df.loc[filas, 'RAD'] = np.nan</code>	asigna NaN en la columna RAD solo en las filas indicadas.
<code>df['RAD'].isin(lista)</code>	True si el valor está en la lista. El ~ delante lo niega (True pasa a False).
<code>X.copy()</code>	hace una copia independiente, para no modificar el DataFrame original al escalar o imputar.
<code>X.to_numpy()</code>	convierte a un array de NumPy (matriz de números sin nombres de columnas).
<code>A @ B</code>	producto de matrices (igual que <code>np.matmul(A, B)</code>).
<code>A.T</code>	matriz transpuesta (filas pasan a columnas).
<code>y.reshape(-1, 1)</code>	convierte un vector de n valores en una matriz de n filas y 1 columna. El -1 significa "calculalo vos".
<code>np.hstack((np.ones((n, 1)), X))</code>	pega una columna de unos a la izquierda de X (sirve para el intercepto, ver sección 10).
<code>np.random.default_rng(42)</code>	generador de números aleatorios con semilla 42: con la misma semilla, siempre salen los mismos números.
<code>np.logspace(-4, 6, 200)</code>	200 valores entre 10^{-4} y 10^6 , repartidos parejo en escala logarítmica.
<code>modelo.coef_</code>	atributo aprendido. En scikit-learn todo lo que termina en guion bajo (<code>coef_</code> , <code>intercept_</code> , <code>alpha_</code> , <code>scale_</code>) existe recién después del <code>fit</code> .

Convención de nombres: **X** (mayúscula) es la matriz de features, porque tiene muchas columnas; **y** (minúscula) es el vector del target. Viene de la notación matemática.

3. La pregunta estrella: fit, transform, fit_transform y predict

scikit-learn organiza todo en objetos que comparten los mismos métodos. Hay dos familias:

- **Transformadores** (cambian X): `RobustScaler`, `StandardScaler`, `KNNImputer`. Tienen `fit`, `transform` y `fit_transform`.
- **Modelos o estimadores** (predicen y): `LinearRegression`, `RidgeCV`, `LassoCV`, `ElasticNetCV`, `CatBoostClassifier`. Tienen `fit` y `predict`.

Método	Qué hace	Qué devuelve	En el TP
<code>fit(X)</code>	Aprende de los datos y lo guarda dentro del objeto. No cambia X.	el mismo objeto, ya entrenado	el scaler aprende mediana e IQR de train
<code>transform(X)</code>	Aplica lo que se aprendió en el fit. No aprende nada nuevo.	X transformado	escalar test con la mediana e IQR de train
<code>fit_transform(X)</code>	fit y transform sobre los mismos datos, en un solo paso.	X transformado	escalar train
<code>fit(X, y)</code>	En un modelo: aprende los parámetros (coeficientes) a partir de X e y.	el modelo entrenado	<code>modelo_lineal.fit(X_train_imp, y_train)</code>
<code>predict(X)</code>	Usa los parámetros aprendidos para estimar y.	las predicciones $\hat{y}$	<code>modelo_lineal.predict(X_test_imp)</code>

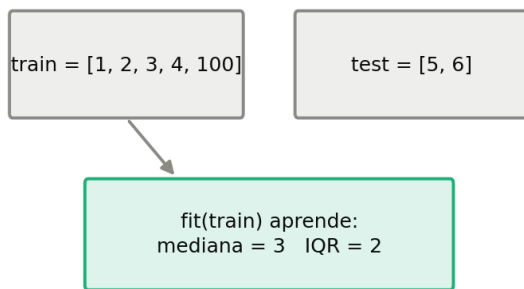
Así aparece en el notebook, en la celda del escalado:

Celda "RobustScaler solo sobre las continuas"

```
scaler = RobustScaler()
# train: aprende mediana e IQR de train y escala train
X_train_esc[columnas_continuas] = scaler.fit_transform(X_train[columnas_continuas])
# test: solo aplica la mediana y el IQR que aprendió de train
X_test_esc[columnas_continuas] = scaler.transform(X_test[columnas_continuas])
```

¿Por qué no hacer `fit_transform` también en test? Porque el escalador volvería a aprender, esta vez con la mediana y el IQR de test. Pasan dos cosas malas: test queda en una escala distinta a la que el modelo aprendió (los mismos números significan cosas distintas), y además usamos información de test para preparar los datos, que es fuga de datos. El Gráfico 2 lo muestra con números:

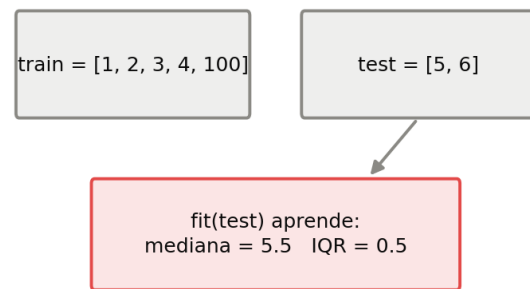
Correcto: fit en train, transform en test



test transformado = [1, 1.5]

fórmula: $(x - \text{mediana}) / \text{IQR}$

Incorrecto: fit también en test



test transformado = [-1, 1]

fórmula: $(x - \text{mediana}) / \text{IQR}$

Gráfico 2. Ejemplo inventado con RobustScaler. A la izquierda, test se escala con la regla de train: el 5 queda en 1.0 (está 1 IQR por encima de la mediana de train). A la derecha, con fit en test, el mismo 5 queda en -1.

Fijate que en el ejemplo correcto el 5 y el 6 quedan por encima de 0, porque son más grandes que la mediana de train (3). En el incorrecto, el 5 quedó negativo: el modelo lo leería como "un valor bajo", cuando en realidad es alto.

Todos los lugares del TP donde aparece este patrón

Objeto	Con train	Con test
RobustScaler (continuas)	fit_transform	transform
KNNImputer(n_neighbors=2)	fit_transform	transform (completa test con vecinos de train)
CatBoostClassifier (RAD y CHAS)	fit con las filas que tienen el dato, predict para los faltantes	predict con el mismo modelo, sin reentrenar
StandardScaler (solo para el descenso del gradiente)	fit_transform	transform
LinearRegression	fit y predict	predict
Pipelines make_pipeline(StandardScaler(), RidgeCV(...))	fit (adentro hace fit_transform del scaler y fit del modelo)	predict (adentro hace transform y predict)

Para la defensa

Si te preguntan la diferencia entre `fit` y `fit_transform`: `fit` solo aprende (por ejemplo, calcula mediana e IQR) y no devuelve los datos transformados; `fit_transform` aprende y además transforma esos mismos datos. Se usa en train. En test se usa solo `transform`, para aplicar lo aprendido en train sin volver a aprender.

Una línea que lo resume: *fit aprende la regla, transform la aplica, predict la usa para predecir.*

4. Limpieza de los datos

4.1 Filas sin target

MEDV tiene 21 nulos. Esas filas se eliminan con `df_casas.dropna(subset=['MEDV'])`. No se imputan porque el target es justamente lo que el modelo tiene que aprender: rellenarlo sería inventar las respuestas del examen. Quedan 535 filas.

4.2 Valores inválidos de RAD

Celda de limpieza de RAD

```
df_casas_limpio['RAD'] = df_casas_limpio['RAD'].round()
valores_validos_rad = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 24.0]
df_casas_limpio.loc[~df_casas_limpio['RAD'].isin(valores_validos_rad), 'RAD'] = np.nan
```

Línea por línea: primero redondea (10.55 pasa a 11). Después arma la lista de categorías válidas. Por último, en las filas donde RAD **no** está en esa lista (por eso el ~), pone NaN. Así un 11 o un 15 quedan como faltantes y se imputan más adelante con CatBoost. RAD queda con 23 nulos.

4.3 Filas casi vacías

Se cuenta cuántos nulos tiene cada fila. De las 535, 506 están completas. Con `dropna(thresh=len(columnas)*0.5)` (o sea, `thresh=7`) se conservan las filas con al menos 7 valores reales de 14. Salen 4 filas que tenían 8 o más nulos: tendrían más valores inventados que medidos. El dataset queda en **531 filas**.

5. Análisis exploratorio

5.1 Media, mediana y asimetría

La **media** es el promedio; la **mediana** es el valor del medio cuando ordenás los datos. Si hay unos pocos valores enormes, la media se va hacia ellos y la mediana casi no se mueve. Cuando media y mediana están lejos, la distribución es **asimétrica**. El número que lo mide es el **skew** (asimetría): 0 es simétrica, positivo es cola hacia la derecha, negativo cola hacia la izquierda.

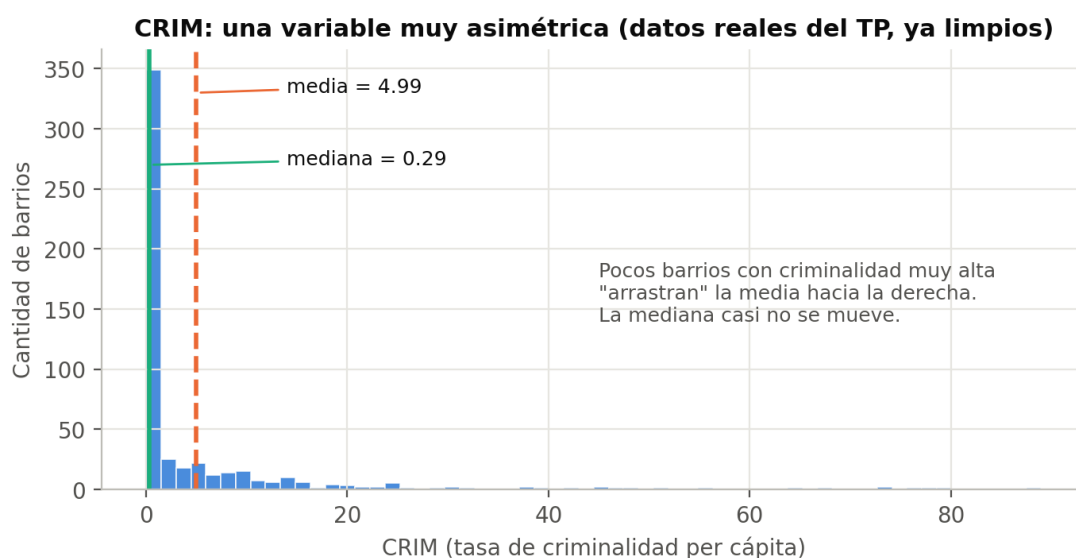


Gráfico 3. Datos reales: CRIM tiene media 4.99 y mediana 0.29 (skew 4.14).

En el TP casi todas las continuas son asimétricas: CRIM (skew 4.14), ZN (2.07), B (-2.57), DIS y MEDV (1.07), LSTAT (0.96). Solo INDUS (0.30) y RM (0.26) son casi simétricas. MEDV además tiene un pico en 50: 16 barrios valen exactamente 50, lo que parece un tope del dataset.

5.2 Tests de normalidad

Muchas herramientas (el z-score, el coeficiente de Pearson) funcionan mejor si los datos tienen forma de campana (distribución normal). Para chequearlo usamos los tests de **Shapiro-Wilk** y **D'Agostino-Pearson**.

- Los dos parten de la hipótesis "los datos son normales".
- Devuelven un **p-valor**: la probabilidad de ver datos tan "raros" si de verdad fueran normales.
- Si $p < 0.05$ (el α del notebook), rechazamos la normalidad.

En las 12 continuas (11 features y MEDV) el p-valor dio prácticamente 0 en los dos tests: **ninguna es normal**. Esa conclusión decide tres cosas del TP: usamos IQR en vez de z-score para outliers, Spearman como referencia para correlaciones y RobustScaler para escalar.

5.3 Boxplots, IQR y outliers

Q1 es el valor que deja abajo al 25 % de los datos y Q3 al 75 %. El $IQR = Q3 - Q1$ es el ancho de la caja. Un valor es outlier si cae por debajo de $Q1 - 1.5 \cdot IQR$ o por encima de $Q3 + 1.5 \cdot IQR$. El 1.5 es el factor estándar (**IQR_FACTOR = 1.5** en el notebook).

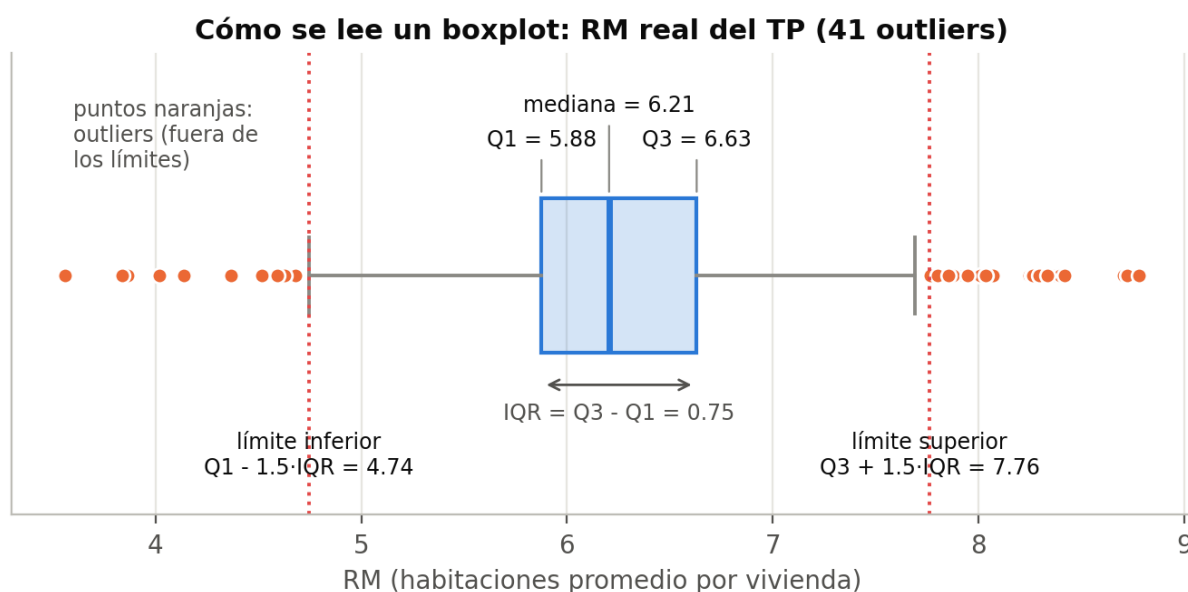


Gráfico 4. Datos reales: lectura del boxplot de RM, con los mismos límites que calcula el notebook.

Variable	B	CRIM	ZN	RM	MEDV	LSTAT	DIS	INDUS, NOX, AGE, TAX, PTRATIO
Outliers IQR	86	64	54	41	37	11	10	0

¿Por qué no usar z-score? El z-score suele marcar como outlier lo que está a más de 3 desvíos de la media. Pero la media y el desvío se calculan con los mismos outliers que queremos detectar, y además el criterio supone normalidad, que acá no se cumple. Los

cuartiles casi no se mueven por unos pocos extremos.

5.4 Correlación: Pearson y Spearman

Pearson mide qué tan bien los puntos se acomodan a una recta. **Spearman** reemplaza cada valor por su posición (1.º, 2.º, 3.º...) y después calcula Pearson sobre esas posiciones: mide si "cuando una sube, la otra también sube", sin importar cuánto. Por eso un valor extremo para Spearman es solo "el más grande" y pesa poco.

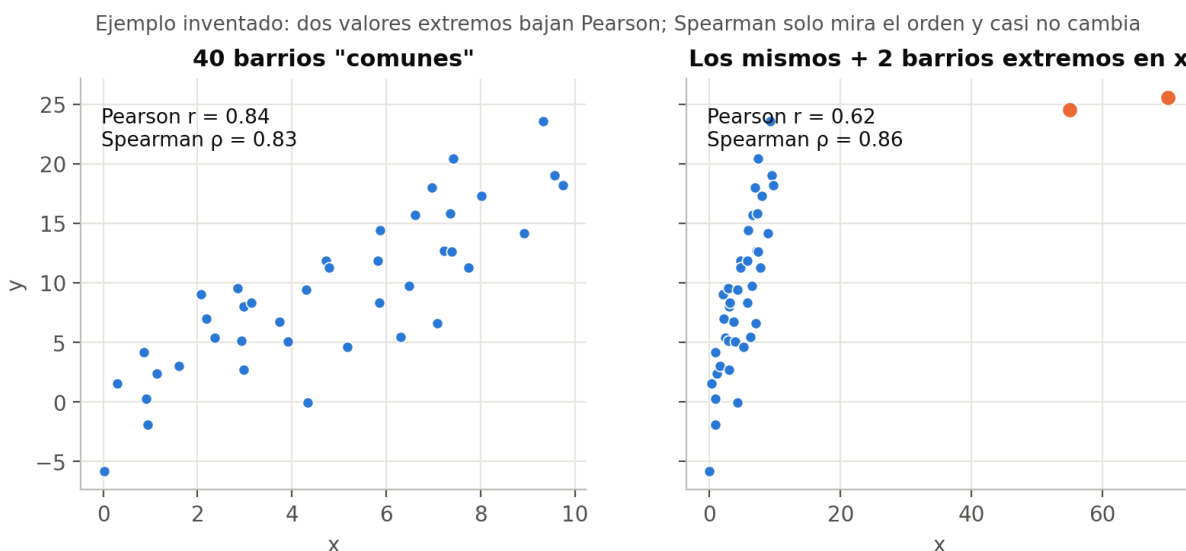


Gráfico 5. Ejemplo inventado: dos puntos extremos en x bajan Pearson de 0.84 a 0.62; Spearman pasa de 0.83 a 0.86.

Resultados del TP (Spearman como referencia, porque los datos no son normales y tienen outliers):

- Con MEDV, la relación más fuerte es **LSTAT** (-0.80): más población de bajo estatus, menor precio. Le sigue **RM** (0.58): más habitaciones, mayor precio.
- **RAD y TAX**: 0.90 con Pearson y 0.70 con Spearman. Es **multicolinealidad**: las zonas más urbanizadas tienen impuestos más altos y mejor acceso a autopistas, así que las dos variables cuentan casi la misma historia.
- **CRIM** parece aislada con Pearson ($\text{CRIM-ZN} = 0.01$) pero con Spearman aparece relacionada ($\text{CRIM-NOX} = 0.80$, $\text{CRIM-TAX} = 0.72$). Es el mismo efecto del Gráfico 5: pocos barrios extremos distorsionan Pearson.

5.5 Codificación de las variables categóricas

La consigna pide decidir cómo codificar las categóricas si se usan para predecir.

- **CHAS** ya viene como 0 y 1. No necesita transformación.
- **RAD** es un índice **ordinal**: un valor más alto indica mejor acceso a autopistas. En la regresión la dejamos numérica. La alternativa era **one-hot encoding** (una columna 0/1 por cada categoría), que sumaba 8 columnas nuevas, varias con muy pocas filas (por ejemplo, $\text{RAD} = 7$ tiene 18 barrios).
- RAD solo se trata como categórica al **imputar**: ahí no se puede promediar, porque saldrían valores como 10.5 que no existen en el índice.

6. Outliers: ¿error o caso raro pero real?

El IQR marcó muchos outliers en CRIM, ZN y B, pero esas variables son muy asimétricas, así que la mayoría son la cola natural de la distribución. Borrarlos todos habría tirado información real. En lugar de eso, el notebook revisa la **coherencia** de cada outlier con lo que dice la matriz de correlación.

Cómo funciona chequear_coherencia

1. Para cada variable se definen los signos esperados. Por ejemplo, para CRIM: relación negativa con MEDV (más crimen, menor precio), negativa con RM y positiva con LSTAT.
2. Para cada outlier se mira si está del lado alto o bajo, y si las otras variables de esa fila están por encima o por debajo de su mediana en la dirección esperada.
3. Se cuenta cuántas relaciones "rompe". Si rompe al menos la mitad de las que se pueden evaluar (`umbral_fraccion=0.5`), el outlier es **incoherente**. Los NaN no se evalúan: solo achican el denominador.

Ejemplo: un barrio con CRIM altísimo (outlier alto) debería tener MEDV bajo, RM bajo y LSTAT alto. Si tiene MEDV alto, RM alto y LSTAT bajo, rompe 3 de 3 relaciones: es sospechoso.

Resultado: 10 incoherentes de 64 en CRIM, 6 de 54 en ZN y 21 de 86 en B. La **triangulación** busca las filas incoherentes en al menos dos variables, que son candidatas mucho más fuertes a error: fueron 8 (14, 23, 264, 298, 316, 458, 520 y 554). Se eliminan solo las que cayeron en train (7). La 520 quedó en test y se conserva, porque test tiene que parecerse a los datos reales que llegarían "sin limpiar".

7. Separar train y test (y por qué antes de escalar e imputar)

Celda del split

```
X = df_casas_limpio.drop(columns='MEDV') # features
y = df_casas_limpio['MEDV']             # target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- `test_size=0.2`: 20 % para test. De 531 filas: 424 en train y 107 en test. Después de sacar 7 filas trianguladas, **train queda con 417**.
- `random_state=42`: la mezcla es aleatoria, pero con semilla fija siempre sale igual. Así cualquiera que corra el notebook obtiene los mismos resultados.
- Devuelve cuatro cosas, siempre en ese orden: X de train, X de test, y de train, y de test.
- Mantiene el índice original de cada fila. Por eso después se puede preguntar si la fila 520 está en `X_train.index` o en `X_test.index`.

¿Por qué el split va **antes** de escalar e imputar? Porque el escalador y los imputadores aprenden algo de los datos (mediana, IQR, vecinos, un clasificador). Si los entrenamos con todo el dataset, la información de test ya estaría adentro del preprocesamiento: fuga de datos. Separando primero, todo lo que aprende lo hace solo con train.

Un detalle que conviene saber

La exploración y el análisis de outliers (secciones 5 y 6) usaron el dataset completo, antes del split. Por eso las filas trianguladas se sacan solo de train: test no se "limpia" con decisiones tomadas mirando sus propios valores.

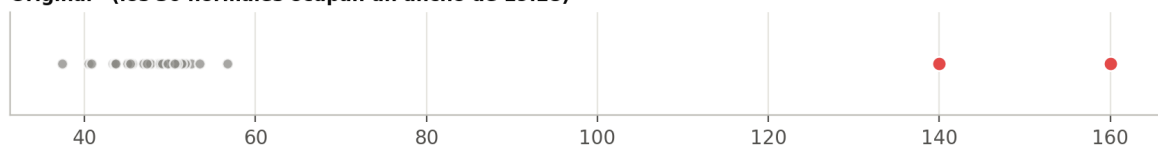
8. Escalado

Las variables viven en escalas muy distintas: TAX va de 187 a 711 y NOX de 0.38 a 0.87. Sin escalar, en una distancia (KNN) TAX taparía por completo a NOX, y en el descenso del gradiente o la regularización una misma tasa o un mismo alpha afectaría muy distinto a cada variable.

Escalador	Fórmula	Usa	Problema con outliers
StandardScaler	$(x - \text{media}) / \text{desvío estándar}$	media y desvío	los outliers inflan el desvío y aplastan al resto de los datos
RobustScaler	$(x - \text{mediana}) / \text{IQR}$	mediana e IQR	casi no lo afectan

Ejemplo inventado: 30 valores normales y 2 outliers (rojo)

Original (los 30 normales ocupan un ancho de 19.28)



StandardScaler: $(x - \text{media}) / \text{desvío}$ (los 30 normales ocupan un ancho de 0.77)



RobustScaler: $(x - \text{mediana}) / \text{IQR}$ (los 30 normales ocupan un ancho de 3.55)

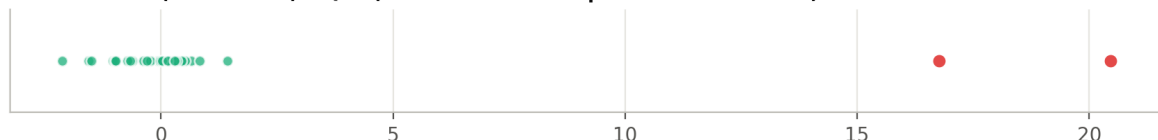


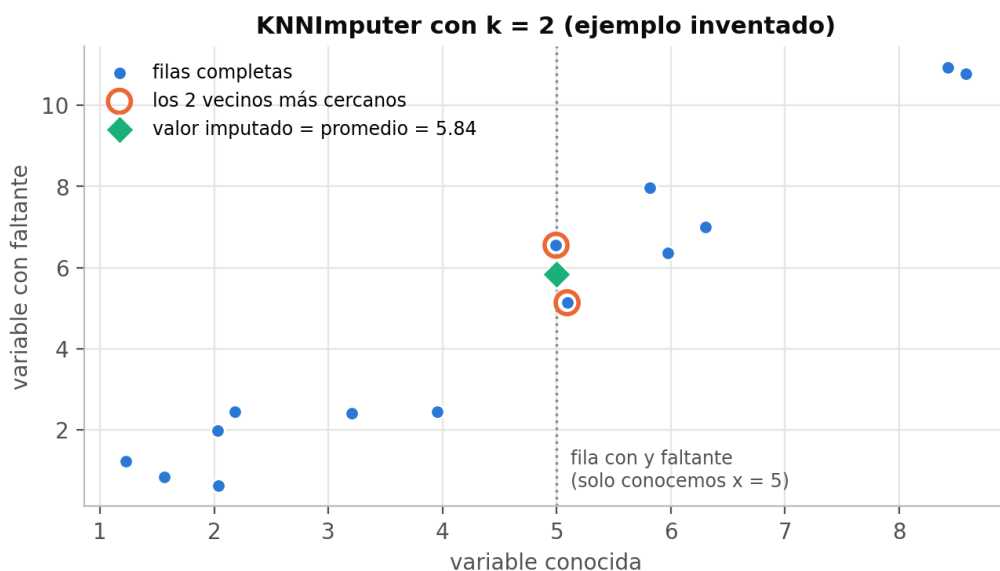
Gráfico 6. Ejemplo inventado: con StandardScaler los 30 valores normales quedan amontonados en un ancho de 0.77; con RobustScaler conservan un ancho de 3.55.

En el TP se usa **RobustScaler** sobre las 11 continuas, porque decidimos conservar muchos outliers legítimos (CRIM, ZN, B). CHAS y RAD no se escalan. Más adelante aparece **StandardScaler** dos veces: solo para el descenso del gradiente (sección 11) y dentro de los pipelines de regularización (sección 13).

9. Imputación de valores faltantes

9.1 KNNImputer para las continuas

KNN significa "k vecinos más cercanos". Para rellenar un valor faltante, el imputador busca las k filas de train más parecidas a esa fila (usando las columnas que sí tiene) y pone el **promedio** del valor en esos vecinos. Con $k = 2$, promedia 2 vecinos.



Por eso importa escalar antes: "más cercano" se mide con una distancia, y sin escalar las variables grandes dominarían.

9.2 Cómo se eligió k (método del codo)

k es un hiperparámetro. Para elegirlo sin tocar test, el notebook hace un experimento de **enmascaramiento**:

1. Toma las 405 filas de train que están completas (sabemos todos sus valores reales).
2. Oculta al azar el 5 % de las celdas (las pone en NaN).
3. Las imputa con un k dado y compara lo imputado con lo real. El error es el **NRMSE**: un RMSE dividido por el desvío de cada columna, para que todas las variables pesen parecido.
4. Repite 15 veces (con distintas semillas) para cada k en 1, 2, 3, 4, 5, 7, 10, 15, 20, 25 y 30, y promedia.

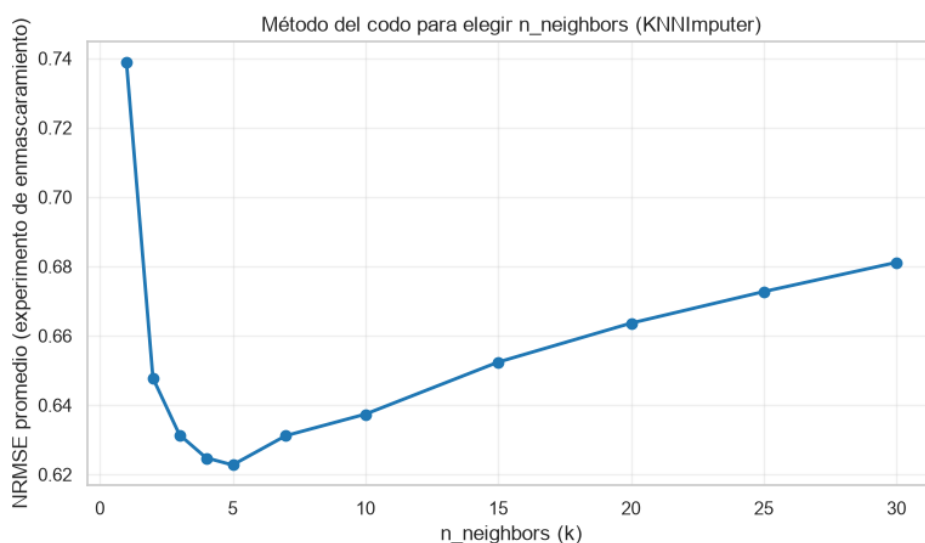


Figura 10. Método del codo: NRMSE promedio vs. $n_neighbors$ (k) para KNNImputer.

Gráfico 8. Figura 10 del notebook: NRMSE promedio según k .

k	1	2	3	4	5	7	10	30
NRMSE	0.739	0.648	0.631	0.625	0.623	0.631	0.637	0.681

El error baja mucho de 1 a 2 vecinos, sigue bajando poco hasta $k = 5$ (mínimo) y después sube, porque con muchos vecinos se promedian filas poco parecidas. El "codo" es donde la curva cambia de bajar fuerte a aplanarse. En vez de elegirlo a ojo, el notebook calcula la **derivada segunda** (cuánto cambia la pendiente de un tramo al siguiente) y elige el k donde es mayor en valor absoluto: $k = 2$ (0.075, contra 0.010 en $k = 3$).

Si te preguntan "¿por qué $k = 2$ si el mínimo está en $k = 5$?"

Porque el criterio elegido fue la curvatura (el codo), no el mínimo. La diferencia de NRMSE entre $k = 2$ y $k = 5$ es 0.025, chica comparada con el desvío entre repeticiones (0.063 en $k = 2$). Lo reconocemos en el notebook: $k = 5$ daría un error de imputación algo menor.

9.3 CatBoostClassifier para CHAS y RAD

Promediar vecinos funciona para variables continuas, pero no para categorías: el promedio de un 0 y un 1 en CHAS es 0.5, que no existe. Por eso CHAS y RAD se imputan como un problema de **clasificación**: un modelo aprende a predecir la categoría a partir de las demás columnas.

Celda de CatBoost (resumida)

```
for col in ['RAD', 'CHAS']:
    # todas las columnas menos la que imputamos
    features = [c for c in X_train_imp.columns if c != col]
    # filas de train que SÍ tienen el dato
    conocido = X_train_imp[X_train_imp[col].notna()]
    modelo = CatBoostClassifier(iterations=500, learning_rate=0.05, depth=4,
                                verbose=False, random_seed=42,
                                allow_writing_files=False)
    modelo.fit(conocido[features], conocido[col].astype(int))
    # después: predict para los faltantes de train y de test, con el mismo modelo
```

- Se entrena un clasificador por columna, solo con las filas de train que tienen el dato.
- `.astype(int)`: un clasificador necesita las clases como etiquetas (24.0 pasa a 24).
- `.flatten()` (en la línea del predict): CatBoost devuelve las predicciones como matriz de $n \times 1$; flatten las pasa a un vector común para poder asignarlas a la columna.
- **Complemento:** CatBoost es un modelo de *gradient boosting*: combina muchos árboles de decisión chicos (acá, de profundidad 4), donde cada árbol corrige los errores de los anteriores. Su `learning_rate` no tiene nada que ver con el lr del descenso del gradiente de la sección 11.

Para verificar, se comparan las distribuciones de train antes y después de imputar (Figuras 12.1 a 12.13 del notebook): casi no cambian, porque en train quedaban entre 0 y 8 valores por imputar en cada variable, y en CHAS la proporción de 0 y 1 se mantiene.

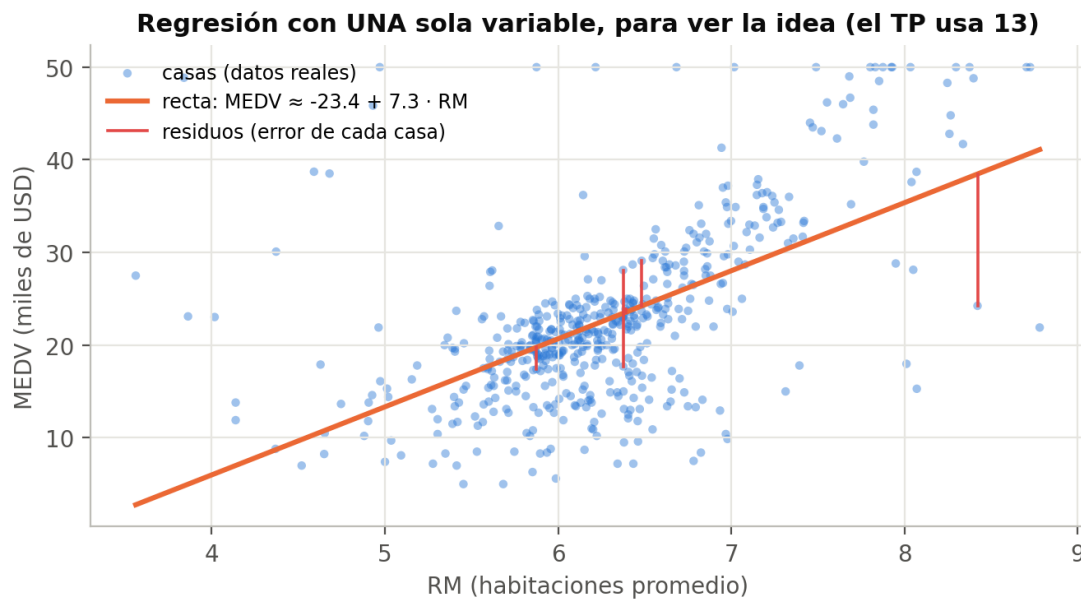
10. Regresión lineal

La regresión lineal predice el precio como una suma ponderada de las features:

$$\hat{y} = w_0 + w_1 \cdot \text{CRIM} + w_2 \cdot \text{ZN} + \dots + w_{13} \cdot \text{LSTAT}$$

$\hat{y}$ ("y sombrero") es la predicción; w_0 es el intercepto (el valor base) y cada w_j dice cuánto cambia la predicción si la feature j aumenta en una unidad y las demás quedan fijas.

`LinearRegression` encuentra los w que minimizan el MSE de forma **exacta**, con una fórmula cerrada (mínimos cuadrados), sin iterar.



Resultado del TP (`LinearRegression` sobre `X_train_imp`): intercepto 22.20. Los coeficientes más grandes son CHAS (4.93), LSTAT (-4.36), DIS (-3.52), AGE (-2.80), PTRATIO (-2.68), RM (2.66) y TAX (-2.28).

- CHAS = 4.93: a igualdad de lo demás, limitar con el río suma unos 4.930 dólares al valor estimado.
- Los signos tienen sentido: más LSTAT, menor precio; más RM, mayor precio.
- Ojo: las continuas están escaladas con `RobustScaler` y CHAS y RAD no. Un coeficiente de una continua dice cuánto cambia el precio por cada IQR que sube esa variable, así que no se puede comparar directamente con el de CHAS.

La forma matricial y la columna de unos

Para no escribir la suma a mano, se arma una matriz X (una fila por casa, una columna por feature) y un vector de pesos W . Entonces todas las predicciones juntas son $X @ W$. El intercepto se resuelve con un truco: se agrega una columna de unos al principio de X . Así el primer peso multiplica siempre a 1 y funciona como w_0 :

```
# columna de unos a la izquierda: X pasa de 13 a 14 columnas
X_tr = np.hstack((np.ones((n, 1)), X_tr))
# W tiene 14 pesos: el primero es el intercepto
y_pred = X_tr @ W
```

11. Descenso del gradiente

El descenso del gradiente busca el mínimo del MSE sin fórmula cerrada, de a pasos. Imaginá que estás en una montaña con niebla y querés llegar al valle: mirás hacia dónde baja más el terreno bajo tus pies y das un paso para ese lado. Repetís hasta que ya no baja.

11.1 La fórmula del notebook

Costo: $MSE = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$, con $\hat{y} = X \cdot W$.

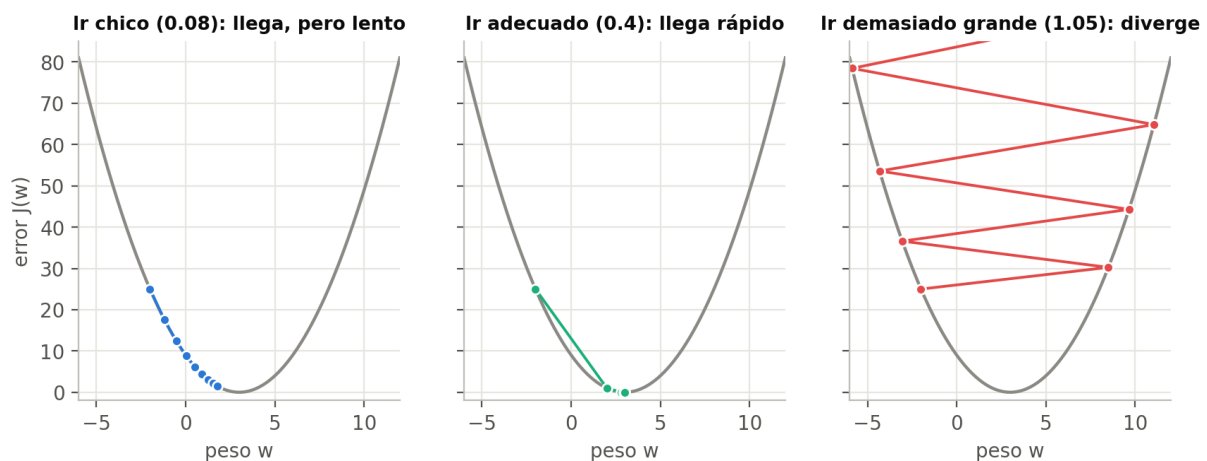
Gradiente (hacia dónde sube el costo): $\text{gradiente} = -(2/n) \cdot X^T \cdot (y - X \cdot W)$.

Actualización (paso hacia abajo): $W = W - lr \cdot \text{gradiente}$.

Corazón de descenso_gradiente_batch

```
residuo = y_tr - X_tr @ W          # cuánto se equivoca en cada fila
gradiente = -2 / n * (X_tr.T @ residuo) # dirección en la que el error crece
W = W - lr * gradiente              # paso en la dirección contraria
```

La **tasa de aprendizaje (lr)** es el tamaño del paso. Si es chica, llega pero tarda; si es grande, llega rápido; si es demasiado grande, se pasa del mínimo cada vez más lejos y el error explota (diverge).



11.2 Las tres variantes

Variante	Cuándo actualiza W	Pasos por época en el TP	lr usado	Características
Batch	una vez por época, con el gradiente de todo train	1	0.1	la más estable; tolera la tasa más alta
Estocástico (SGD)	una vez por cada fila	417	0.0005	rápido pero ruidoso; necesita tasa muy chica
Mini-batch	una vez por lote de filas	7 (lotes de 64)	0.01	punto medio; el más usado en la práctica

Camino hacia el mínimo (ejemplo inventado, curvas de nivel del error)

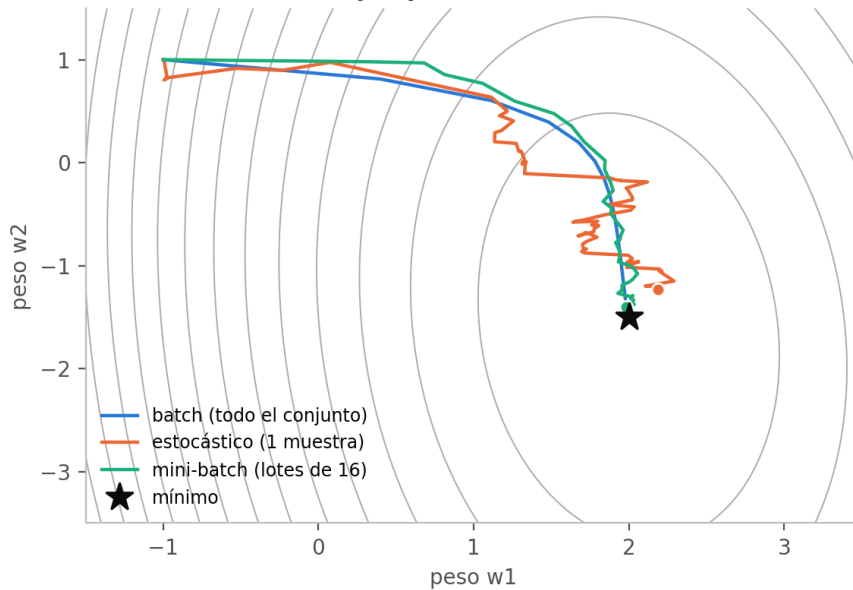


Gráfico 11. Ejemplo inventado: el batch baja suave; el estocástico zigzaguea porque cada paso mira una sola fila; el mini-batch queda en el medio.

Regla para recordar: **cuantas más actualizaciones por época, más chica tiene que ser la tasa**, porque cada paso usa menos datos y es más ruidoso.

11.3 Detalles del código que suelen preguntar

- **¿Por qué StandardScaler solo para el descenso del gradiente?** RAD (hasta 24) y CHAS (0 o 1) no pasaron por RobustScaler. Con escalas tan mezcladas, una sola tasa no sirve para todas las variables: para algunas es demasiado grande y para otras demasiado chica. Se estandariza todo, se entrena y después se vuelven los pesos a la escala original con `recuperar_parametros`: $\text{coeficiente} = W_j / \text{desvío}_j$ e $\text{intercepto} = W_0 - \sum W_j \cdot \text{media}_j / \text{desvío}_j$.
- `reshape(-1, 1)` en `y`: `X @ W` da una matriz de $n \times 1$. Si `y` fuera un vector plano de n valores, la resta `y - X @ W` haría "broadcasting" y daría una matriz de $n \times n$, un error silencioso. Con `y` también en $n \times 1$, la resta es fila a fila.
- **Pesos iniciales al azar** con `np.random.default_rng(semilla).standard_normal(...)`: el punto de partida es aleatorio pero reproducible.
- **Corte del batch**: frena cuando el MSE de una época a la siguiente cambia menos que `tolerancia=1e-9`. El estocástico y el mini-batch oscilan, así que corren un número fijo de épocas.
- **Mezcla en cada época** (`rng.permutation(n)`): en estocástico y mini-batch las filas se reordenan al azar para que los lotes no sean siempre los mismos.
- **El error de test** se guarda solo para mirarlo; nunca se usa para actualizar los pesos.

11.4 Resultados

Variante	Épocas	MSE final en train
Batch (lr 0.1)	461 (cortó por tolerancia)	32.6660
Estocástico (lr 0.0005)	100	32.6859
Mini-batch (lr 0.01, lote 64)	300	32.6715

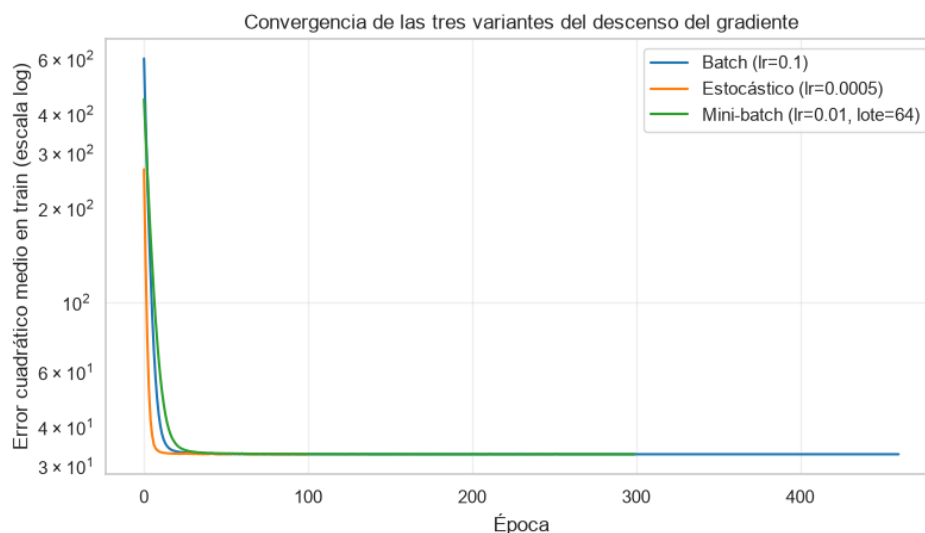


Figura 13. MSE de entrenamiento por época para las tres variantes.

Gráfico 12. Figura 13 del notebook: las tres curvas se aplanan cerca de la época 50.

Las tres llegan prácticamente a la misma solución que LinearRegression. El batch coincide hasta el cuarto decimal en las métricas. La razón: el MSE de la regresión lineal es una función **convexa** (forma de cuenco), con un único mínimo, así que cualquier camino que baje bien termina en el mismo lugar. Con 417 filas da igual cuál usar; el descenso del gradiente se vuelve necesario cuando hay tantos datos que la fórmula exacta es demasiado costosa.

11.5 Las funciones de la cátedra y "¿Algún cambio?"

La consigna pide gráficas de error contra iteraciones. El notebook usa las funciones del material de clase (`gradient_descent`, `stochastic_gradient_descent`, `mini_batch_gradient_descent`) sin cambiar el cálculo, con sus parámetros: 200 épocas, $lr = 0.01$ y lotes de 11 (38 lotes por época). Lo que se observa:

- Batch: baja suave pero no llega al mínimo en 200 épocas. Por eso en nuestra versión usamos $lr = 0.1$.
- Mini-batch: converge rápido.
- Estocástico: oscila sin asentarse (son unas 83.400 actualizaciones). Necesita una tasa mucho menor, como el 0.0005 de nuestra versión. Además su curva de train mide una sola fila por vez, así que es ruidosa por construcción.

12. Métricas: cómo medimos el error

Métrica	Fórmula (en palabras)	Unidad	Se lee como
MAE	promedio de $ \text{real} - \text{predicho} $	miles de USD	error típico, sin castigar extra a los grandes
MSE	promedio de $(\text{real} - \text{predicho})^2$	miles de USD al cuadrado	difícil de interpretar (por eso no la usamos)
RMSE	raíz cuadrada del MSE	miles de USD	error típico, castigando más los errores grandes
R^2	$1 - (\text{error del modelo} / \text{error de predecir siempre el promedio})$	sin unidad	proporción de la variabilidad explicada (1 es perfecto)

Ejemplo inventado con 3 casas. Precios reales 20, 30 y 25; predicciones 22, 27 y 25 (miles de USD). Errores: -2, 3 y 0.

- $MAE = (2 + 3 + 0) / 3 = 1.67$
- $MSE = (4 + 9 + 0) / 3 = 4.33$, y $RMSE = \sqrt{4.33} = 2.08$ (más que el MAE porque el error de 3 pesa más al elevarse al cuadrado)
- R^2 : el promedio real es 25. Si siempre predijéramos 25, el error cuadrado total sería $25 + 25 + 0 = 50$. El del modelo es $4 + 9 + 0 = 13$. $R^2 = 1 - 13/50 = 0.74$.

Se descartaron dos métricas de la lista de la consigna: el **MSE**, porque es el RMSE al cuadrado y está en unidades raras, y el **MAPE** (error porcentual), porque en las casas baratas un error chico en dólares se vuelve un porcentaje enorme.

¿Por qué medir en train y en test?

Train dice qué tan bien el modelo ajusta lo que ya vio; test, qué tan bien predice casas nuevas. Comparar los dos es el diagnóstico:

Train	Test	Diagnóstico
bueno	mucho peor	sobreajuste (memorizó train)
malo	malo	subajuste (modelo demasiado simple)
bueno	parecido	buen ajuste

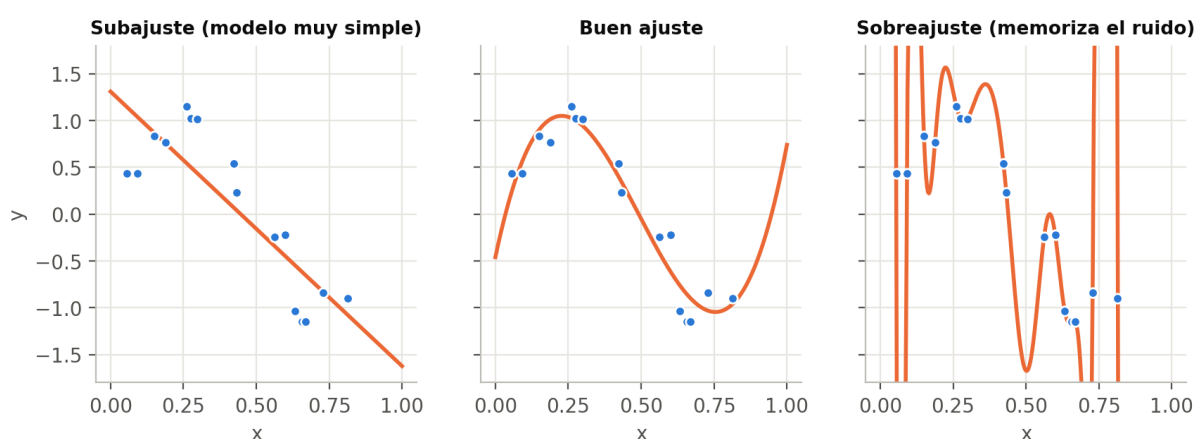


Gráfico 13. Ejemplo inventado: la recta no alcanza a seguir la forma (subajuste); el polinomio de grado 15 pasa por todos los puntos y se vuelve loco entre ellos (sobreajuste).

¿Conseguimos un buen fitting?

LinearRegression: R^2 de 0.611 en train y 0.617 en test; RMSE de 5.72 y 6.43. Train y test dan casi igual, así que **no hay sobreajuste**. Pero sí hay **algo de subajuste**: queda sin explicar cerca del 40 % de la variabilidad, y en test el modelo se equivoca en promedio unos 4.200 dólares (MAE) o 6.400 (RMSE). Causas probables: relaciones que no son lineales y el tope de 50 en MEDV. El notebook aclara que no se comprobó con un gráfico de residuos, así que queda como hipótesis.

13. Regularización: Ridge, Lasso y ElasticNet

La regularización le suma al costo un castigo por el tamaño de los coeficientes. El modelo tiene que equilibrar dos cosas: predecir bien y tener coeficientes chicos. Sirve para frenar el sobreajuste, sobre todo cuando hay variables correlacionadas.

Modelo	Costo (forma simplificada)	Efecto
Ridge (L2)	$MSE + \alpha \cdot \sum w_j^2$	achica todos los coeficientes, ninguno llega a 0
Lasso (L1)	$MSE + \alpha \cdot \sum w_j $	achica y puede dejar coeficientes en 0 exacto (elige variables)
ElasticNet	mezcla de los dos castigos	<code>l1_ratio</code> decide la mezcla: 0 es Ridge, 1 es Lasso

(Fórmulas simplificadas para entender la idea; scikit-learn usa constantes un poco distintas delante de cada término.) El intercepto no se castiga. **alpha (α)** es la fuerza del castigo: con $\alpha = 0$ es la regresión lineal común; con α muy grande, todos los coeficientes van a 0 y el modelo predice siempre el promedio.

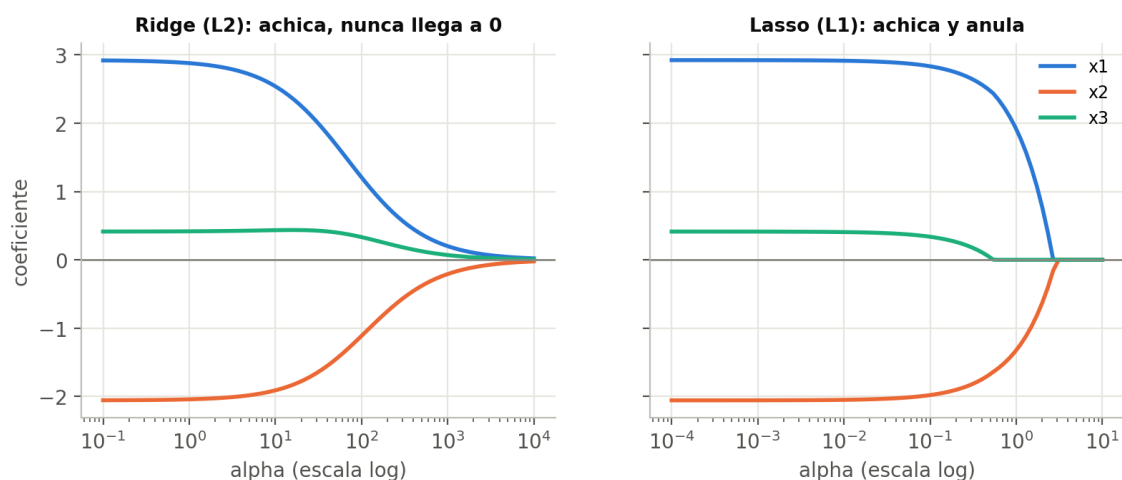


Gráfico 14. Ejemplo inventado con 3 variables: al subir α , Ridge achica de forma suave; Lasso anula primero la variable menos importante (x3).

Por qué hay que escalar antes

El castigo mira el tamaño de los coeficientes, y ese tamaño depende de las unidades de cada variable. Una variable medida en números grandes tiene un coeficiente chico y casi no se castigaría. Por eso cada modelo va en un pipeline con `StandardScaler`:

Celda de regularización

```
modelos_reg = {
    'Ridge': make_pipeline(StandardScaler(),
                           RidgeCV(alphas=np.logspace(-4, 6, 200))),
    'Lasso': make_pipeline(StandardScaler(),
                           LassoCV(alphas=np.logspace(-4, 2, 200), cv=5)),
    'ElasticNet': make_pipeline(StandardScaler(), ElasticNetCV(
        l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.95, 0.99],
        alphas=np.logspace(-3, 6, 200), cv=10)),
}
```

- `make_pipeline` encadena pasos. Al hacer `fit`, el scaler hace `fit_transform` con `train` y el modelo entrena sobre eso; al hacer `predict`, el scaler solo transforma. Es el patrón de la sección 3, automático.

- `modelo[-1]` es el último paso del pipeline (el modelo). Por eso se lee `modelo[-1].alpha_`.
- Los coeficientes de estos modelos quedan en escala estandarizada: se comparan entre sí, no con los de LinearRegression.

Validación cruzada: cómo se elige alpha sin tocar test

La diferencia entre `Ridge` y `RidgeCV` es que la versión CV recibe una lista de alphas, prueba cada uno con validación cruzada dentro de train y al final reentrena con el mejor sobre todo train.

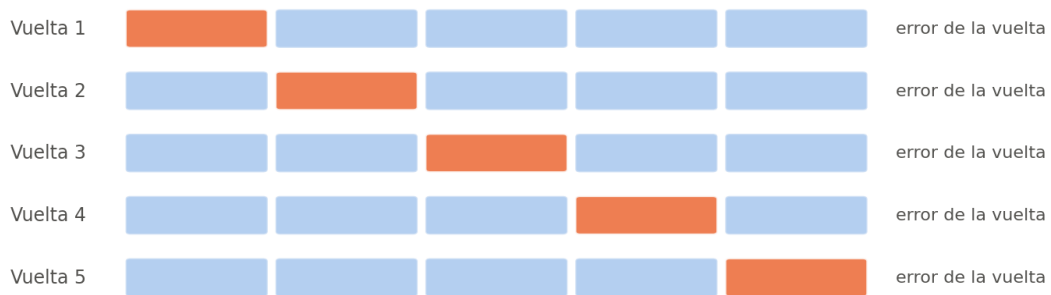


Gráfico 15. Validación cruzada con 5 partes (folds): cada parte valida una vez y entrena las otras cuatro.

- LassoCV usa 5 folds (`cv=5`) y ElasticNetCV 10.
- RidgeCV, por defecto, usa *leave-one-out*: cada fila valida una vez. Para Ridge hay una fórmula que lo hace rápido.
- Se sacó `l1_ratio=0` de ElasticNet porque equivale a Ridge (ya lo probamos) y daba problemas de convergencia.

Resultados

Modelo	alpha elegido	Coeficientes en 0	RMSE train	RMSE test	R ² test
LinearRegression	(no tiene)	ninguno	5.7154	6.4267	0.6174
Ridge	67.48	ninguno	5.7791	6.5089	0.6076
Lasso	0.111	CRIM, NOX, RAD	5.7393	6.4793	0.6112
ElasticNet	0.1482 (l1_ratio 0.5)	NOX, RAD	5.7602	6.4774	0.6114

Ninguno mejora a LinearRegression en test. Tiene sentido: la regularización corrige el sobreajuste, y este modelo no sobreajusta. Lo que sí hace Lasso es simplificar: anula RAD, que estaba muy correlacionada con TAX (información repetida).

14. Optimización de hiperparámetros

Se varía un hiperparámetro por vez, dejando el resto fijo, y se mira qué pasa.

14.1 Tasa de aprendizaje del batch

El notebook calcula el límite teórico: $\mathbf{l}_{\max} = 2 / \lambda_{\max}$, donde $\lambda_{\max}$ es el autovalor más grande de $(2/n) \cdot X^T X$. En palabras simples: esa matriz describe qué tan curvado (qué tan cerrado) es el "valle" del error en cada dirección, y $\lambda_{\max}$ es la curvatura de la dirección más

cerrada. Si el paso supera $2 / \lambda_{\max}$, en esa dirección cada paso cae del otro lado más arriba que antes y el error explota (como el panel derecho del Gráfico 9, donde la curvatura es 2 y el límite es 1). Con los datos del TP da **0.176**.

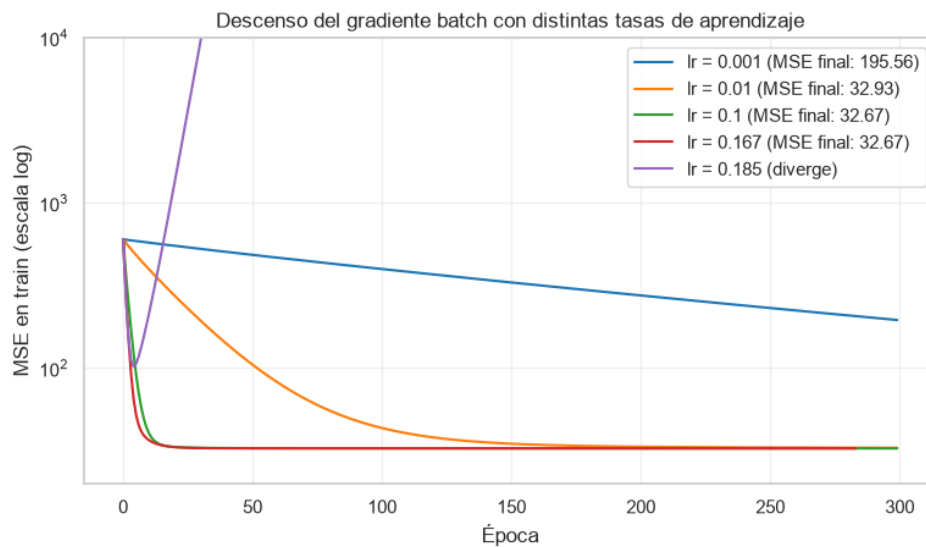


Figura 17. MSE de entrenamiento por época según la tasa de aprendizaje (300 épocas como máximo).

Gráfico 16. Figura 17 del notebook: MSE por época con distintas tasas (300 épocas como máximo).

lr	0.001	0.01	0.1	0.167 (95 % del límite)	0.185 (105 % del límite)
MSE a las 300 épocas	195.56 (no llega)	32.93	32.67 (en unas 30 épocas)	32.67	diverge

Conclusión: la mejor tasa es la más grande que todavía converge, y ese límite se puede calcular con los datos.

14.2 Tamaño del lote

Con $lr = 0.01$ y 100 épocas fijas. Un lote de 1 es el estocástico; uno de 417 (todo train) es el batch.

Lote	1	8	32	128	417
MSE train	36.93	32.98	32.91	32.82	43.37
MSE test	43.45	41.92	42.18	42.55	56.44

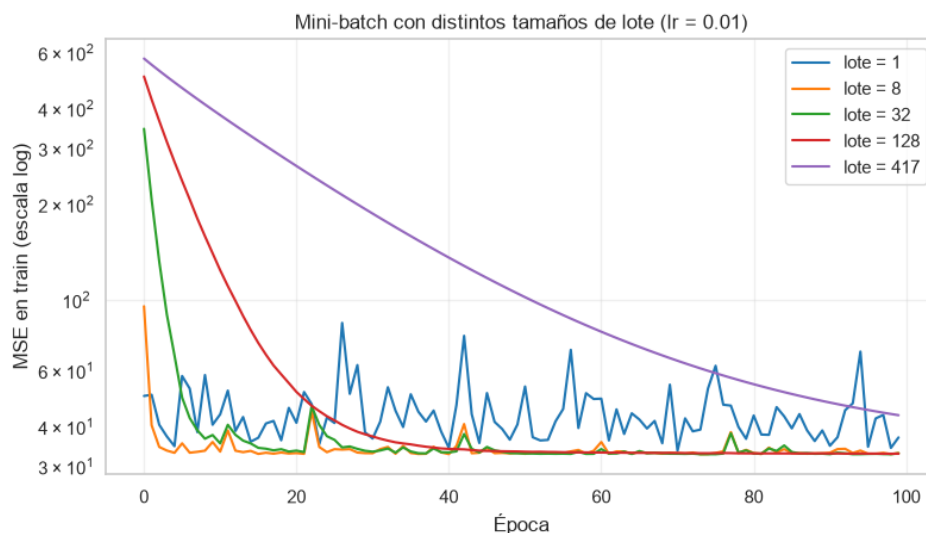


Figura 18. MSE de entrenamiento por época según el tamaño del lote.

Gráfico 17. Figura 18 del notebook: MSE por época según el tamaño del lote.

Los lotes chicos actualizan más veces por época y bajan más rápido; el lote completo (417) hace una sola actualización por época y no llega a converger en 100. Pero el lote de 1 es tan ruidoso que queda rebotando alrededor del mínimo. Los intermedios equilibran los dos extremos.

14.3 alpha en Ridge y Lasso

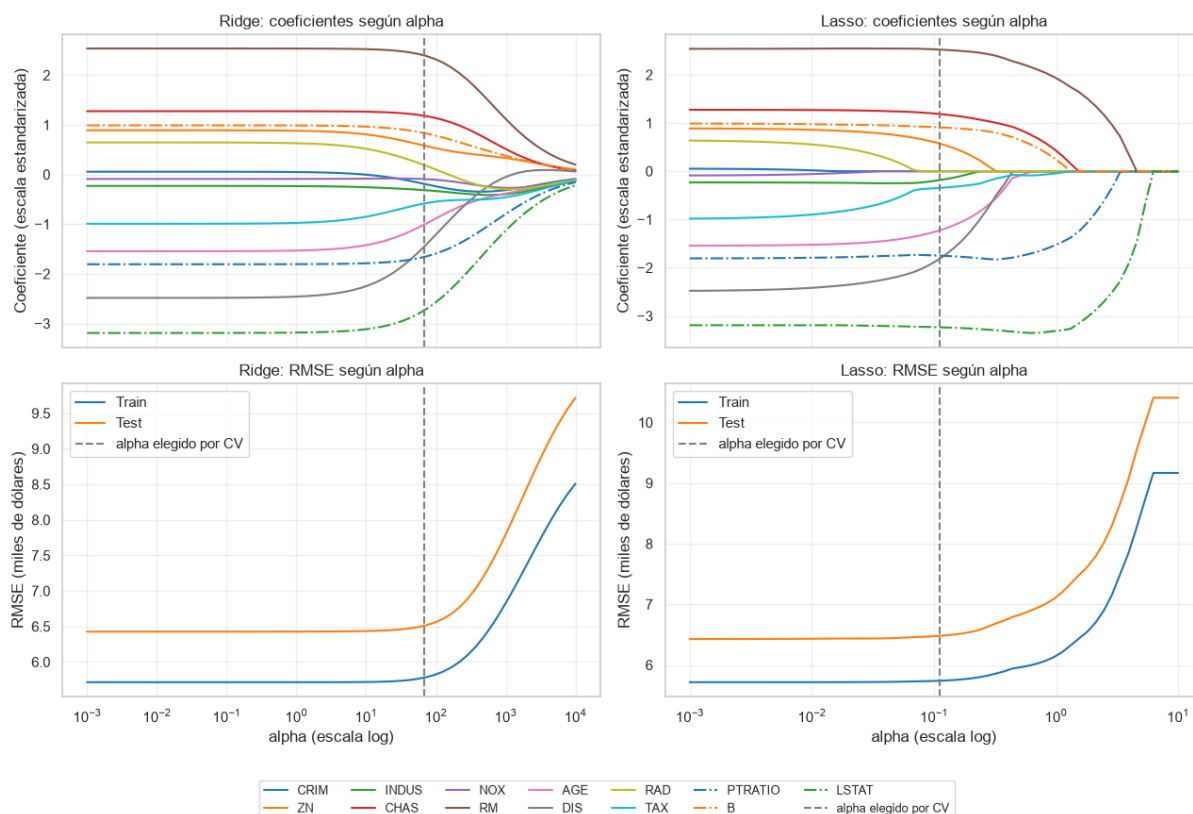


Figura 19. Coeficientes y RMSE de train y test en función de alpha, para Ridge y Lasso.

Gráfico 18. Figura 19 del notebook: coeficientes (arriba) y RMSE de train y test (abajo) según alpha. La línea punteada es el alpha de la validación cruzada.

- Con alpha chico, Ridge y Lasso equivalen a la regresión lineal (zona plana a la izquierda).

- Al subir alpha, Ridge achica todos los coeficientes sin anular ninguno; Lasso los va anulando de a uno.
- El RMSE de train sube siempre. El de test **nunca baja**: no aparece la curva en U que veríamos si hubiera sobreajuste (primero mejora al regularizar y después empeora).
- Es el **compromiso sesgo-varianza**: regularizar cambia varianza por sesgo. Como acá no sobra varianza, solo se suma sesgo.

15. Comparación final y conclusión

Para comparar se usa el **RMSE en test**: test porque son casas que ningún modelo vio, y RMSE porque está en miles de dólares y castiga más los errores grandes, que en un precio son los que más importan. El R^2 ordena igual a los modelos sobre el mismo test, así que no agrega información.

Modelo	MAE test	RMSE test	R^2 test
LinearRegression	4.2010	6.4267	0.6174
GD Batch	4.2010	6.4267	0.6174
GD Estocástico	4.1891	6.4359	0.6163
GD Mini-batch	4.2114	6.4559	0.6140
ElasticNet	4.2167	6.4774	0.6114
Lasso	4.2354	6.4793	0.6112
Ridge	4.2078	6.5089	0.6076

- LinearRegression y GD Batch empatan: son la misma solución, una exacta y otra por aproximaciones sucesivas.
- Entre el mejor y el peor (Ridge) hay 0.08 de RMSE, unos 80 dólares sobre un error típico de 6.400: en la práctica los siete predicen igual.
- **Elegimos LinearRegression**: tiene el mejor resultado, es exacta, no tiene hiperparámetros que ajustar ni aleatoriedad. Con diferencias tan chicas, otra partición de train y test podría cambiar el orden, así que la elección se apoya más en la simplicidad que en los decimales.
- Para mejorar haría falta un modelo más flexible (términos polinómicos o interacciones entre variables), no más regularización.

16. Preguntas sobre el código que te pueden hacer

Respuestas cortas, para decir con tus palabras.

¿Qué diferencia hay entre `fit()` y `fit_transform()`?

`fit` aprende de los datos (por ejemplo, la mediana y el IQR) y lo guarda en el objeto, sin devolver nada transformado. `fit_transform` aprende y además devuelve esos mismos datos transformados. Se usa en train; en test se usa solo `transform`.

¿Y entre `transform()` y `predict()`?

`transform` lo tienen los transformadores (escaladores, imputadores) y devuelve X modificado. `predict` lo tienen los modelos y devuelve predicciones del target.

¿Por qué no se hace fit con test?

Porque sería fuga de datos: el preprocesamiento usaría información de las casas "del futuro" y las métricas de test saldrían optimistas. Además test quedaría en otra escala que la que aprendió el modelo.

¿Para qué sirve random_state=42?

Fija la semilla del azar para que el split (y otros procesos aleatorios) den siempre lo mismo. El 42 es arbitrario; lo importante es que sea fijo.

¿Por qué se hace el split antes de escalar e imputar?

Porque escalador e imputadores aprenden de los datos. Si se ajustaran con todo el dataset, ya conocerían test.

¿Por qué RobustScaler y no StandardScaler?

Porque conservamos muchos outliers legítimos y RobustScaler usa mediana e IQR, que no se dejan arrastrar por los extremos. StandardScaler usa media y desvío, que sí.

¿Por qué CHAS y RAD se imputan con CatBoost y no con KNN?

Porque son categóricas: un promedio de vecinos daría valores que no existen (CHAS = 0.5, RAD = 10.5). Un clasificador siempre devuelve una categoría válida.

¿Por qué se eliminan las filas sin MEDV en lugar de imputarlas?

Porque MEDV es el target: imputarlo sería inventar las respuestas que el modelo tiene que aprender.

¿Qué hace dropna(thresh=7)?

Conserva solo las filas con al menos 7 valores no nulos de 14. Eliminó 4 filas con 8 o más nulos.

¿Para qué la columna de unos (np.hstack)?

Para que el primer peso de W funcione como intercepto: multiplica siempre a 1.

¿Qué es una época? ¿Y la tasa de aprendizaje?

Una época es una pasada completa por train. La tasa es el tamaño de cada paso del descenso del gradiente.

¿Por qué cada variante del descenso del gradiente usa una tasa distinta?

Porque hacen distinta cantidad de actualizaciones por época (1, 7 y 417). Con más actualizaciones, cada una usa menos datos y es más ruidosa, y una tasa grande la hace oscilar o diverger.

¿Por qué el descenso del gradiente llega a lo mismo que LinearRegression?

Porque el MSE de la regresión lineal es convexo y tiene un único mínimo. LinearRegression lo calcula exacto; el descenso del gradiente se acerca de a pasos.

¿Qué diferencia hay entre Ridge y RidgeCV?

Ridge usa un alpha que le das vos. RidgeCV prueba una lista de alphas con validación cruzada en train, se queda con el mejor (alpha_) y reentrena con él.

¿Por qué los atributos terminan en guion bajo (coef_, alpha_)?

Es la convención de scikit-learn para lo que se aprende en el fit. Antes del fit no existen.

¿Qué hace make_pipeline?

Encadena pasos. En el fit, el scaler hace fit_transform y el modelo fit; en el predict, el scaler solo transforma. Garantiza que el escalado se aprenda solo con los datos de entrenamiento.

¿Por qué el RMSE y no el MSE para comparar?

Porque el RMSE está en miles de dólares, como MEDV, y se puede interpretar; el MSE está al cuadrado.

¿Por qué la regularización no mejoró el modelo?

Porque corrige el sobreajuste y este modelo no sobreajusta (train y test dan casi igual). Solo sumó sesgo.

Ejemplo Práctico / Analogía

El concepto más difícil: fit en train, transform en test.

Pensá en un profesor que tiene que poner notas con "curva". Con los parciales de su curso del año pasado (**train**) arma la regla: "la nota del medio fue 60 puntos, y la mitad central del curso se reparte en 20 puntos; entonces cada 20 puntos por encima de 60 es un escalón más de nota". Armar esa regla es el **fit**: mirar los datos y sacar la mediana y el IQR.

Aplicarle la regla a esos mismos parciales es el **transform**; hacer las dos cosas de una es el **fit_transform**. Cuando llega un examen nuevo (**test**), el profesor usa **la misma regla**: un 80 sigue siendo "un escalón por encima del medio". Si para corregir el examen nuevo armara otra regla con las notas de ese examen, un 80 podría quedar "por debajo del medio" solo porque ese día a todos les fue bien. Las notas dejarían de ser comparables con las del año anterior, que es exactamente lo que le pasa al modelo cuando se hace fit en test (Gráfico 2).

Y el **predict**? Es lo que hace el modelo después: con las notas ya convertidas con la regla de train, estima el resultado final del alumno.

Dónde falla la analogía

En la analogía, armar una regla nueva con el examen nuevo parece solo "injusto". En machine learning el problema es doble: los datos de test quedan en otra escala que la que aprendió el modelo, y además el preprocesamiento usa información de test, así que las métricas dejan de medir cómo le iría al modelo con casas realmente nuevas.

Con los números del TP en la cabeza. RobustScaler se ajusta con las 417 filas de train de cada variable continua. Cuando llega una casa de test, su valor de CRIM se convierte con la mediana y el IQR de CRIM de train. Lo mismo pasa con KNNImputer (los vecinos se buscan en train) y con CatBoost (el clasificador se entrenó con train y solo predice en test).

Autoevaluación Rápida

Intentá responder sin mirar lo anterior. Si alguna te cuesta, volvé a la sección indicada.

1. ¿Por qué el RobustScaler se usa con **fit_transform** en train y solo con **transform** en test? ¿Qué pasaría con un valor de test si hiciéramos fit también en test? (sección 3)
2. Si los tests de normalidad no hubieran rechazado la normalidad, ¿qué decisiones del preprocesamiento podrían haber sido distintas? (secciones 5 y 8)
3. ¿Por qué el descenso del gradiente llega a la misma solución que LinearRegression, y por qué el estocástico necesita una tasa mucho más chica que el batch? (sección 11)
4. Lasso anuló RAD. ¿Qué tiene que ver eso con la correlación entre RAD y TAX? (secciones 5 y 13)
5. ¿Por qué la regularización no mejoró el RMSE de test en este TP, y qué habría que ver en train y test para que sí ayudara? (secciones 12 y 14)